Análise do impacto de Oracles em diferentes níveis na acurácia de Sistemas de Múltiplos Classificadores

Relatório de resultados — respostas aos objetivos específicos

Subprojeto de Iniciação Científica

28 de agosto de 2026

Resumo

Este relatório responde, um a um, aos oito objetivos específicos do subprojeto, a partir de um experimento de 29 bases × 10 folds estratificados × 5 modos de geração de pool (1450 folds, 0 erros), com pools de $M = 100$ classificadores. Três dos cinco modos separam o efeito do classificador-base do efeito do método de geração — um confounder de versões anteriores deste relatório — e um deles reativa a votação de medidas de complexidade do PGDCS completo, antes desligada. As medidas Oracle_1 a Oracle_M foram implementadas e verificadas; a monotonicidade $\text{Oracle}_1 \geq \text{Oracle}_2 \geq \cdots \geq \text{Oracle}_M$ vale em 1450/1450 folds, assim como $\text{Oracle}_1 \geq$ votação majoritária. Os limites foram confrontados com a acurácia individual, a votação majoritária, a média de probabilidades e *doze* métodos de seleção dinâmica de classificadores e de *ensembles*.

Os cinco resultados centrais são: **(i)** o eixo que organiza os resultados é o classificador-base, não o método de geração — votar as medidas de complexidade do PGDCS ou buscar com o algoritmo genético não muda Oracle_1, MVR ou DF/e^2 de forma estatisticamente sustentada ($p \geq 0.13$ em todos os contrastes), e o padrão inteiro (Oracle_1 alto, MVR baixo, folga grande) reaparece igual sem busca alguma, some apenas quando o classificador-base muda para árvore; **(ii)** nenhum nível intermediário de Oracle_N serve como limite superior realista *nos problemas binários* — o menor N em que Oracle_N cai abaixo da votação majoritária fica em $N \approx M/2$ nos cinco modos (mediana 52–53, Friedman $p = 0.48$), porque aí vale $\text{Oracle}_{M/2+1} \leq MVR \leq \text{Oracle}_{M/2}$, verificado em 1100 de 1100 folds binários; **(iii)** a folga $\text{Oracle}_1 - MVR$ é previsível *antes* de treinar qualquer método de seleção, pelo índice de redundância de erros DF/e^2 (Spearman -0.89, $n = 145$), validado fora da amostra (85,5% de acerto *leave-one-dataset-out* contra 75,9% da regra trivial); **(iv)** mesmo tomando o melhor de dez métodos dinâmicos escolhido *a posteriori* no teste, a seleção dinâmica recupera entre 14,2% e 24,3% dessa folga, conforme o modo — o Oracle_1 é um limite inatingível na prática, não uma meta; e **(v)** uma replicação do PGDCS sob o protocolo do artigo que o propõe (2 perfis, 27 e 28 bases, árvore de decisão) mostra que a vantagem de 4–6 pontos ali publicada é *viés de seleção* — o número do PGDCS é o máximo de 20 gerações medido no mesmo conjunto que escolheu a geração, o do Bagging não passa por seleção alguma; medidos os dois em teste, o ganho real é de +0,39 ponto (22/27, $p = 0{,}003$) e some quando a validação é pequena e disjunta. O relatório fecha com oito recomendações operacionais.

Sumário

1 Escopo e material **2**

1.1 Protocolo 2

1.2 Definições usadas 3

2 Respostas aos objetivos específicos **4**

2.1 Objetivo 1 — o Oracle como limite superior em DCS/DES 4

2.2 Objetivo 2 — implementação do Oracle tradicional 4

2.3 Objetivo 3 — generalização para Oracle$_N$ 4
2.4 Objetivo 4 — construção dos pools 5
2.5 Objetivo 5 — avaliação experimental nas bases 7
2.6 Objetivo 6 — comparação com métodos reais 7
2.7 Objetivo 7 — níveis intermediários são um limite mais realista? 11
2.8 Objetivo 8 — recomendações e redução de custo 13

3 Replicação do PGDCS contra a fonte **15**
3.1 O gap, e o que o produz 15
3.2 O tamanho honesto do efeito 16
3.3 Duas correções de implementação encontradas no caminho 17

4 Confronto com os resultados esperados **17**

5 Limitações **18**

Referências **19**

1 Escopo e material

O experimento cujos números este relatório reporta é o run canônico `2026-08-24T09-38-20_5aceb11`, executado no commit `5aceb11`, de árvore limpa (`git_dirty: false`). Toda tabela, figura e teste aqui pode ser reproduzido com

```
python scripts/run_experiment.py -full
python scripts/analyze_results.py results/experiments/<run>
python scripts/analyze_review.py results/experiments/<run>
```

e conferido contra `analysis.txt`, `analysis.json` e `review_analysis.txt` gravados dentro do diretório do run. A análise textual completa, com o encadeamento dos achados, está em `docs/resultados-oracle-n.md`; este documento é o recorte que responde diretamente às perguntas do subprojeto.

Procedência. O run anterior (`2286fcc`) tinha `git_dirty: true` — o código real divergia do commit registrado, e nada publicado a partir dele era reproduzível (ADR 0019). O run canônico substitui-o de árvore limpa. Antes de qualquer análise, os três modos comuns aos dois desenhos (`ga`, `bagging`, `rf`) foram comparados coluna a coluna: 3480 **combinações base×fold×modo×métrica de pool comparadas,** 0 **divergências,** $\max|\Delta| = 0{,}000000000$. O run sujo fica **validado**: todo número já publicado sobrevive e ganha procedência reprodutível.

1.1 Protocolo

Duas bases foram excluídas por um portão de viabilidade explícito: a classe mais rara de Ecoli tem 2 instâncias e a de Glass tem 9, ambas abaixo dos 10 folds, o que impede a estratificação. A exclusão está registrada no manifesto do run com o motivo, e não é um descarte silencioso.

Os cinco modos. Cada um isola uma variável (ADR 0019):

PGDCS completo. Perceptrons lineares, algoritmo genético, e a etapa em que o método *vota* quais medidas de complexidade convêm a cada base (`get_best_types`) ativa — a especificação original do método.

Tabela 1: Protocolo experimental.

Item	Valor
Bases	29 (das 31 do catálogo; Ecoli e Glass excluídas pelo portão de viabilidade)
Validação	10-fold estratificado, `random_state=42`
Partição interna	treino / validação-*fitness* / DSEL, disjuntas
Tamanho do pool	$M = 100$ em todos os modos
Modos de pool	PGDCS completo, PGDCS-F1/T1, bags aleatórios, Bagging, Random Forest
Fusores medidos	14 (MVR, combinação suave, 10 dinâmicos, 2 estáticos)
Métricas	acurácia, precisão e revocação macro, F1 macro, acurácia balanceada
Estatística	Friedman + Nemenyi (protocolo Demšar) e Wilcoxon pareado com Holm
Unidade pareada	a *base* (folds promediados antes do teste)
Total	1450 folds, 0 erros, 16h44 de relógio

PGDCS-F1/T1. Idêntico, mas com as medidas fixas em F1 e T1 em vez de votadas. É o modo já publicado em versões anteriores deste relatório; mantido para comparabilidade e para o critério de aceitação do run acima.

Bags aleatórios. Mesmos bags estratificados de 50%, mesmo Perceptron, mesma seed, **sem** busca genética — a população de geração 0 do próprio GA. Isola o efeito da busca.

Bagging. Árvores de decisão sobre reamostragens *bootstrap*.

Random Forest. Árvores com subamostragem adicional de atributos.

Os contrastes que os cinco modos destravam — e cujos resultados aparecem nas seções seguintes — são: PGDCS completo *vs* PGDCS-F1/T1 (efeito de votar as medidas), PGDCS-F1/T1 *vs* bags aleatórios (efeito da busca do GA), bags aleatórios *vs* Bagging (efeito do classificador-base).

1.2 Definições usadas

Para um pool com M classificadores e uma amostra x,

$$\text{Oracle}_N(x) = \mathbf{1}\left[\sum_{j=1}^{M} \mathbf{1}[\,h_j(x) = y(x)\,] \;\geq\; N\right], \qquad N = 1, \ldots, M, \tag{1}$$

de modo que Oracle_1 é o Oracle tradicional e vale $\text{Oracle}_1 \geq \text{Oracle}_2 \geq \cdots \geq \text{Oracle}_M$.

Duas quantidades derivadas organizam o relatório:

Folga. $\text{Oracle}_1 - MVR$, a parcela do potencial anunciado pelo Oracle que a votação majoritária não converte.

Folga recuperada. $\dfrac{\text{melhor método dinâmico} - MVR}{\text{Oracle}_1 - MVR}$, a fração dessa folga que um método real alcança. O numerador usa o *máximo por fold* entre os dez métodos dinâmicos, escolhido no conjunto de teste: é um *oracle sobre métodos* e portanto uma cota superior generosa do que a seleção dinâmica entrega.

Uma terceira quantidade mede diversidade de forma comparável entre pools de força diferente. O Double Fault bruto (`DF`, a fração de pares que erram juntos) penaliza pools fracos por construção, porque classificadores que erram mais erram juntos mais. Normalizando pelo valor esperado sob erros independentes,

$$\text{DF}/e^2, \qquad e = 1 - \text{acurácia individual média}, \tag{2}$$

obtém-se um índice em que 1 significa erros independentes e valores altos significam redundância. O denominador acima assume taxa de erro igual entre classificadores; a versão exata, $\frac{2}{M(M-1)} \sum_{i<j} e_i e_j$, é usada como verificação de rigor na seção do objetivo 8.

2 Respostas aos objetivos específicos

2.1 Objetivo 1 — o Oracle como limite superior em DCS/DES

Estudar o conceito de Oracle em MCS, com ênfase em sua utilização como medida de limite superior em técnicas de DCS e DES.

O Oracle é o seletor ideal: dada uma amostra, ele escolhe um classificador correto se existir algum no pool. Por isso é usado como cota superior de qualquer método de seleção — nenhum seletor real pode acertar uma amostra que *nenhum* membro do pool acerta. Souza e Cavalcanti (2017) documentam que essa cota é frouxa: técnicas DCS selecionam classificadores competentes para no máximo cerca de 85% das instâncias, mesmo com Oracle de 100% no treino.

O experimento reproduz e quantifica essa frouxidão nas 29 bases. O Oracle_1 médio fica entre 0.9951 e 0.9979 nos cinco modos — ou seja, *praticamente não discrimina entre pools*. A informação está na distância até os métodos reais, não no valor do Oracle. Esta é a motivação empírica direta da generalização proposta pelo subprojeto e o fio condutor das seções seguintes.

2.2 Objetivo 2 — implementação do Oracle tradicional

Implementar a medida tradicional de Oracle.

Implementado em `pos/oracle/`, sobre uma matriz de acertos binária $M \times n$ construída a partir das predições de cada classificador do pool. As verificações que sustentam a corretude:

- $\text{Oracle}_1 \geq$ votação majoritária em 1450/1450 **folds**. O Oracle tradicional nunca subestimou um método real de combinação — o que é uma propriedade necessária, e sua violação indicaria erro na matriz de acertos.
- Identidade de consistência, verificada com erro máximo 5.6×10^{-16}:

$$\frac{1}{M}\sum_{N=1}^{M} \text{Oracle}_N \;\equiv\; \text{acurácia individual média do pool.} \tag{3}$$

  Ela decorre de $\sum_N P(\geq N \text{ corretos})/M = \mathbb{E}[\text{fração de classificadores corretos}]$ e liga a curva à matriz de acertos por dois caminhos independentes.

A identidade tem uma consequência metodológica que vale registrar: a **"área sob a curva Oracle_N" não é uma métrica nova**. Ela é a acurácia média dos classificadores-base sob outro nome, e não deve ser reportada como medida independente.

2.3 Objetivo 3 — generalização para Oracle_N

Propor e implementar a generalização do Oracle em diferentes níveis.

Implementada para todo $N = 1, \ldots, M$, com a curva completa gravada por fold. A relação de monotonicidade $\text{Oracle}_1 \geq \text{Oracle}_2 \geq \cdots \geq \text{Oracle}_M$ — que é a definição da medida, não uma hipótese — foi verificada em 1450/1450 **folds, sem exceção**.

Tabela 2: Níveis de Oracle e métodos reais, média sobre 29 bases × 10 folds.

Modo	Oracle_1	Oracle_2	Oracle_5	Oracle_M	MVR	Comb. suave	Acc. indiv.
PGDCS completo	0.9972	0.9953	0.9897	0.1139	0.7771	0.7752	0.7042
PGDCS-F1/T1	0.9976	0.9963	0.9905	0.1179	0.7784	0.7738	0.7052
Bags aleatórios	**0.9979**	0.9957	0.9901	0.1158	0.7808	0.7768	0.7046
Bagging	0.9951	0.9935	0.9884	**0.3052**	0.8467	0.8466	0.7931
Random Forest	0.9970	**0.9958**	**0.9920**	0.1725	**0.8549**	0.8546	0.7806

A tabela já antecipa o problema central do objetivo 7: entre $N = 1$ e $N = 5$ a curva cai apenas de 0.997 para 0.990, enquanto a votação majoritária está em 0.78–0.85. A faixa conservadora que o subprojeto procurava não está aqui.

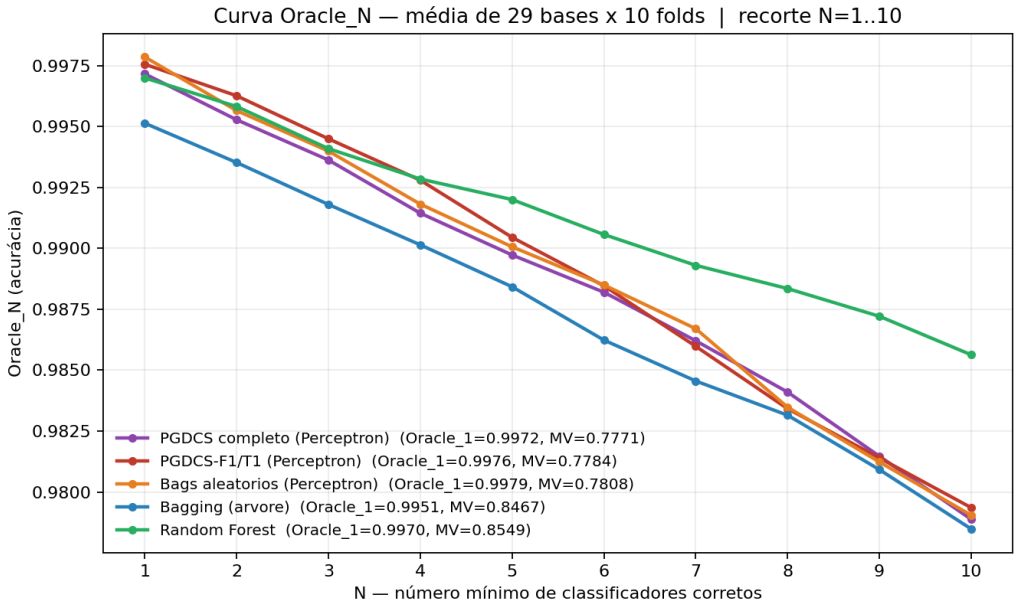


Figura 1: Curva Oracle$_N$ no recorte $N = 1..10$, com a votação majoritária de cada modo. A distância vertical entre a curva e a linha tracejada correspondente é a folga. Os três pools de Perceptron lideram em $N = 1$, mas o Random Forest os ultrapassa a partir de $N = 3$: a vantagem do pool de menor acurácia individual existe só na ponta mais otimista, e a Seção 5 mostra que ela não distingue entre os três métodos de geração — só entre classificadores-base.

2.4 Objetivo 4 — construção dos pools

Construir pools com algoritmos simples e amplamente conhecidos, como Random Forest e Perceptron, induzindo diversidade por variações de treinamento.

Cinco modos, todos com $M = 100$ (protocolo completo na Seção 1):

PGDCS completo. Pool de Perceptrons lineares gerado por algoritmo genético que otimiza medidas de complexidade dos *bags*, com a etapa de votação das medidas (`get_best_types`) ativa — a especificação original do método.

PGDCS-F1/T1. Idêntico, com as medidas fixas em F1 e T1 em vez de votadas por base. É a variante já publicada em versões anteriores deste relatório.

Bags aleatórios. Controle: mesmos bags, mesmo Perceptron, sem busca genética.

Bagging. Árvores de decisão sobre reamostragens *bootstrap*.

Random Forest. Árvores com subamostragem adicional de atributos.

Os cinco atendem ao objetivo (algoritmos simples; diversidade por variação de subconjuntos de dados, de atributos e da própria seleção dos *bags*) e cobrem uma faixa larga de força do classificador-base, o que se mostrou determinante nas seções seguintes.

A medida de diversidade merece destaque porque o número bruto inverte a conclusão:

Tabela 3: Diversidade: o Double Fault bruto contra o índice normalizado.

Modo	Acc. individual	DF bruto	e^2 esperado	DF/e^2
PGDCS completo	0.7042	0.1571	0.0875	2.12
PGDCS-F1/T1	0.7052	0.1566	0.0869	2.13
Bags aleatórios	0.7046	0.1565	0.0873	**2.10**
Random Forest	0.7806	0.1069	0.0481	2.92
Bagging	0.7931	0.1094	0.0428	3.98

Pelo DF bruto os três pools de Perceptron seriam os *menos* diversos (valores mais altos, $\approx$ 0.156–0.157); pelo índice normalizado eles são os *mais* diversos (valores mais baixos, 2.10–2.13; Friedman $p = 0.00026$, $k = 5$). O DF absoluto sozinho leva ao diagnóstico errado, porque classificadores mais fracos erram mais e portanto erram juntos mais.

O confounder geração × classificador-base, resolvido. A versão anterior deste relatório não podia separar "o GA gera diversidade útil" de "o Perceptron é mais fraco individualmente" — os dois efeitos andavam juntos. Comparando os cinco modos par a par (Wilcoxon pareado por base, $N = 29$):

Tabela 4: Contrastes entre modos de geração — nenhuma métrica difere entre PGDCS completo, PGDCS-F1/T1 e bags aleatórios; todas diferem entre bags aleatórios e Bagging.

Contraste	Oracle$_1$	MVR	DF/e^2
PGDCS completo *vs* F1/T1	$p = 0.39$	$p = 0.88$	$p = 0.29$
F1/T1 *vs* bags aleatórios	$p = 0.59$	$p = 0.15$	$p = 0.13$
Bags aleatórios *vs* Bagging	$p = 0.12$	$\mathbf{p = 0.0038}$	$\mathbf{p = 0.0004}$

Nenhuma das duas variáveis de geração dentro do Perceptron — votar as medidas de complexidade ou buscar com o GA — produz uma diferença estatisticamente sustentada em Oracle$_1$, MVR ou DF/e^2. A diferença aparece inteira no contraste seguinte, bags aleatórios *vs* Bagging: mesma ausência de busca dos dois lados, único fator restante é o classificador-base, e é aí que MVR cai 0.066 e DF/e^2 cai de 2.10 para 3.98. **O eixo que organiza os resultados deste relatório é o classificador-base, não o método de geração.** A tese de que a busca do GA por diversidade produz especialistas que a votação desperdiça não se sustenta nestes dados: o mesmo padrão (Oracle$_1$ altíssimo, MVR baixo, folga grande) aparece igual com ou sem busca, e some quando o classificador-base muda para árvore.

Isso também fecha, por medição, se o atalho `F1/T1` custa resultado: não custa. Reativar a votação completa do PGDCS gastou 10,7h de relógio concentradas em duas bases (Magic e WDVG) para produzir um pool estatisticamente indistinguível do que as medidas fixas já entregavam. Curiosamente, a votação raramente escolhe F1/T1: o par aparece em apenas 4 dos 290 folds (1,4%); os pares mais votados são combinações com F3, F4, LSC e N-medidas (Seção 8, objetivo 7). O atalho não é uma boa adivinhação da votação — é uma substituição que, apesar de escolher medidas diferentes, chega ao mesmo resultado.

E a fonte do método, que relata o oposto? A Tabela A.10 de Monteiro et al. (2023) publica ganhos de 4 a 6 pontos do PGDCS sobre Bagging — incompatível com o parágrafo acima. A reconciliação exigiu replicar o protocolo do artigo, com árvore de decisão e as sementes publicadas: o ganho publicado é viés de seleção e o efeito honesto vale +0,39 ponto. Os dois resultados não estavam em conflito; a seção "Replicação do PGDCS contra a fonte" traz a evidência.

2.5 Objetivo 5 — avaliação experimental nas bases

Avaliar o comportamento dos níveis de Oracle em bases públicas de classificação.

Foram usadas 29 bases públicas (UCI, KEEL e as sintéticas clássicas da literatura de MCS: P2, Banana, Lithuanian), com 178 a 19 020 amostras, 2 a 8 classes e razão de desbalanceamento de 1.0 a 71.5.

Ressalva explícita. O objetivo 5 menciona preferência por *bases de imagens* públicas. O catálogo atual privilegia comparabilidade direta com a literatura de referência (as 28 bases de Monteiro et al., 2022, mais Magic) em vez de bases de imagens. É um desvio consciente do enunciado, registrado aqui e nas pendências do `ROADMAP`; incorporar bases de imagens é trabalho futuro imediato, e não altera nenhum dos mecanismos identificados, que dependem da geometria da fronteira e não do domínio.

2.6 Objetivo 6 — comparação com métodos reais

Comparar a acurácia estimada pelos níveis de Oracle com classificadores individuais e métodos reais de combinação (votação majoritária, média de probabilidades) e, quando viável, DCS/DES.

Foram medidos 14 fusores nos 1450 folds dos cinco modos, com DSEL disjunto e $k = 7$ vizinhos ($k = 6$ nos pools de Perceptron, sem os dois métodos que exigem `predict_proba`). Esta seção detalha os três modos já publicados (PGDCS-F1/T1, Bagging, Random Forest); o objetivo 4 (Seção 5) mostra que PGDCS completo e bags aleatórios reproduzem o mesmo padrão do PGDCS-F1/T1 em todas as métricas de pool, sem diferença estatística — a tabela abaixo generaliza para os três modos de Perceptron, não apenas o F1/T1.

Tabela 5: Acurácia média dos 14 fusores contra o teto Oracle_1. Traço = método que exige `predict_proba`, indisponível no Perceptron linear.

Fusor	PGDCS-F1/T1	Bagging	Random Forest
Votação majoritária (MVR)	0.7784	0.8467	0.8549
Combinação suave	0.7738	0.8466	0.8546
OLA	0.7829	0.8028	0.7915
LCA	0.7271	0.7878	0.7863
MCB	0.7828	0.7943	0.7900
Rank	0.7760	0.8022	0.7908
KNORA-E	0.7897	0.8261	0.8274
KNORA-U	**0.8056**	0.8468	**0.8564**
DES-P	0.8021	0.8481	0.8566
DES-KNN	0.7995	0.8477	0.8492
META-DES	—	**0.8482**	0.8545
KNOP	—	**0.8486**	0.8556
Melhor individual (estático)	0.7630	0.7999	0.7958
Seleção estática	0.7797	0.8475	0.8546
Melhor por fold (*oracle* sobre métodos)	0.8340	0.8688	0.8733
Acurácia individual média	0.7052	0.7931	0.7806
Oracle_1	0.9976	0.9951	0.9970
Folga $\text{Oracle}_1 - MVR$	0.2192	0.1484	0.1421
Folga recuperada	**24.0%**	**15.0%**	**14.2%**

(a) O melhor pool produz a pior votação. O PGDCS-F1/T1 tem o *maior* Oracle_1 entre os pools de árvore e Perceptron publicados aqui (0.9976) e a *menor* votação majoritária de todas

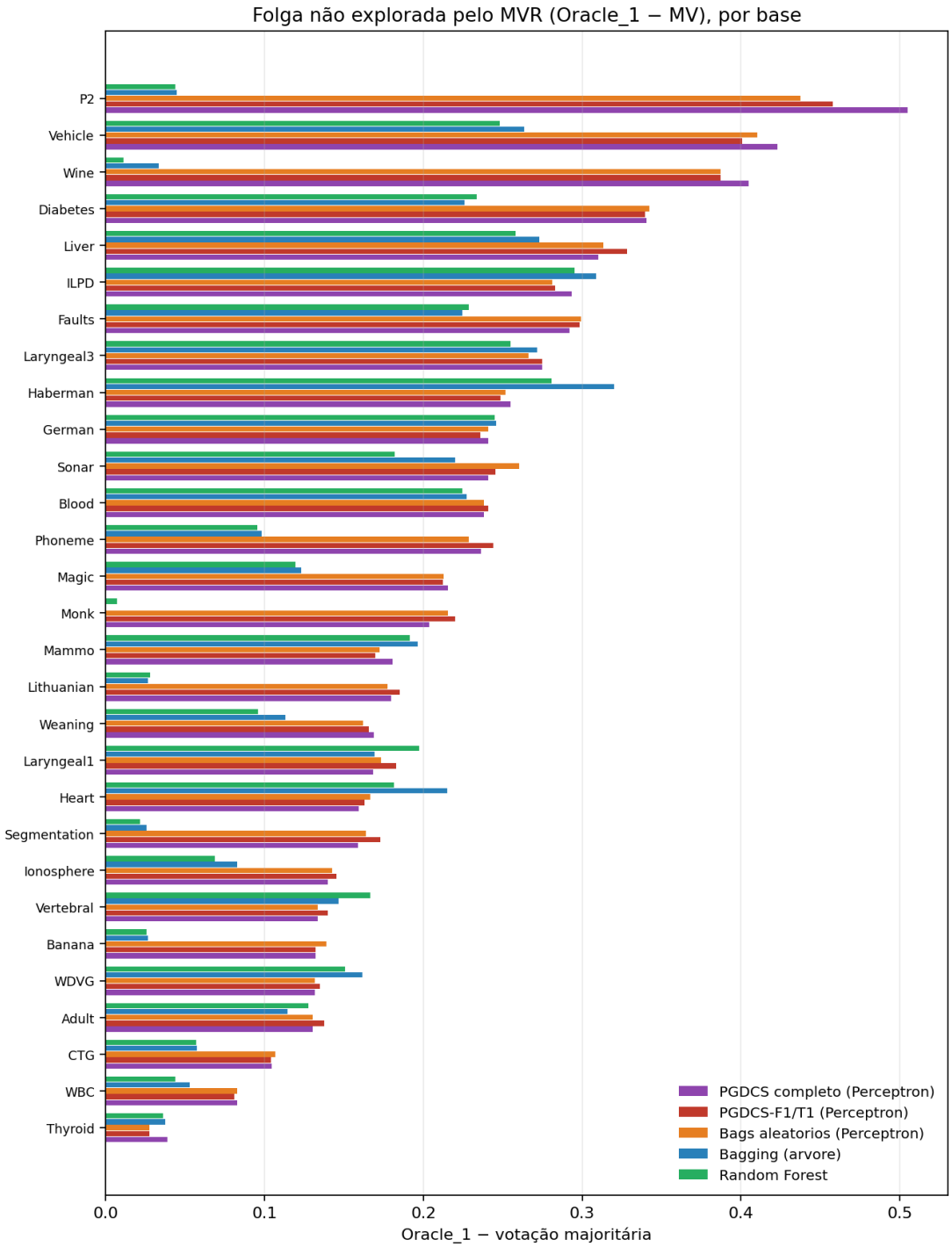


Figura 2: Folga $\text{Oracle}_1 - MVR$ por base. A dispersão entre bases (0.017 a 0.458) é muito maior que a dispersão entre modos (0.142 a 0.220): o potencial não explorado é sobretudo uma propriedade do problema, não da estratégia de geração.

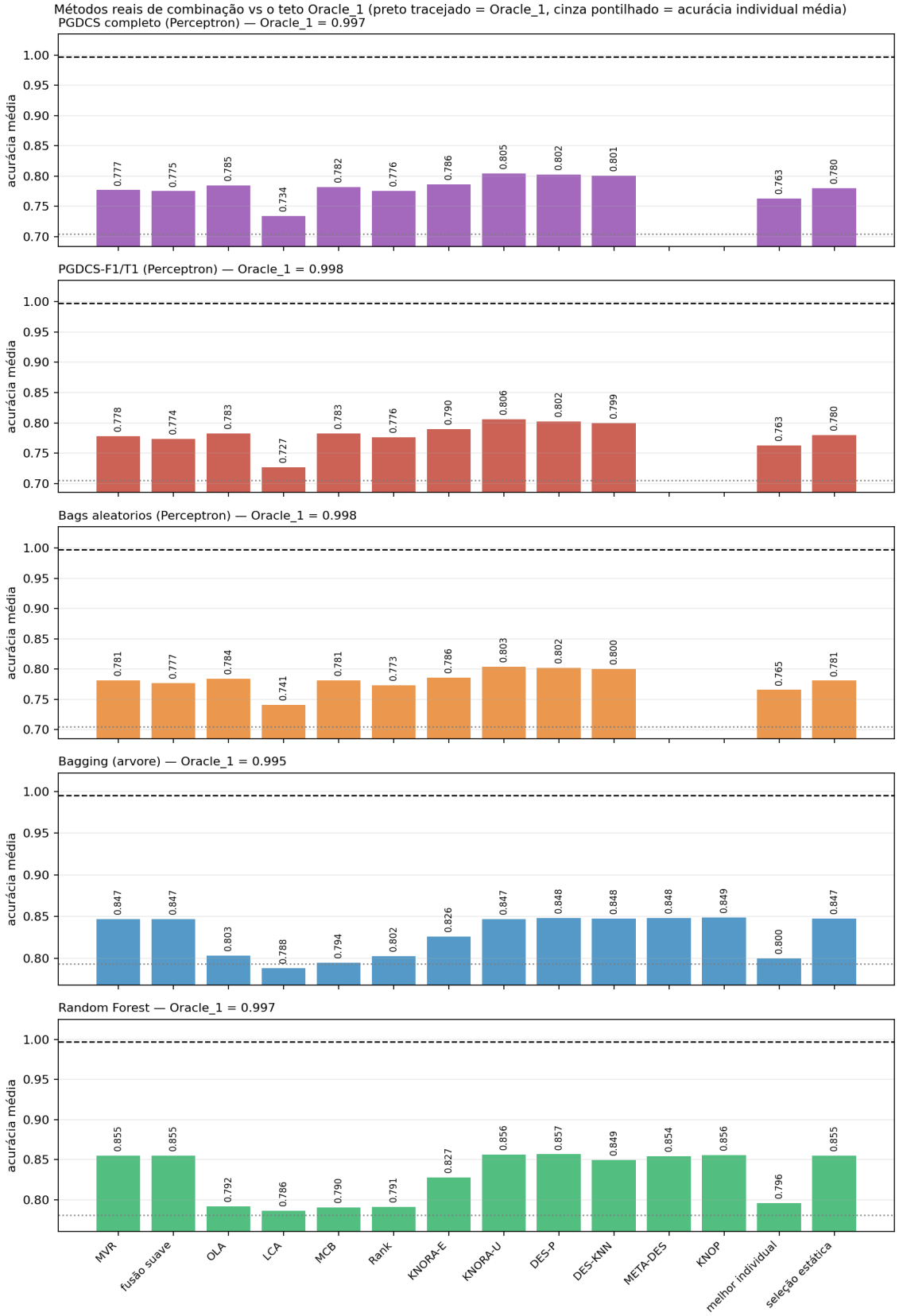


Figura 3: Os 14 fusores contra o teto Oracle_1 (tracejado preto) e a acurácia individual média (pontilhado cinza), um painel por modo. A distância da barra mais alta até a linha tracejada é a parte do potencial do pool que nenhum método alcançou. Barra ausente = método inaplicável ao pool.

(0.7784). A folga resultante, 0.2192, é 48% maior que a do Random Forest (Friedman $p = 0.0022$, $k = 5$; PGDCS-F1/T1 *vs* RF acima da diferença crítica, Wilcoxon $p = 0.0003$). Em P2 o pool contém um classificador correto para *toda* amostra de teste ($\text{Oracle}_1 = 1.0000$), os erros são praticamente independentes ($\text{DF}/e^2 = 1.02$) e a votação majoritária entrega 0.5420. Não é falta de informação no pool — é o fusor que não a extrai.

(b) A combinação suave não supera a votação majoritária em nenhum modo. Nos pools de árvores as duas regras empatam (Δ da ordem de 10^{-4}; dão o mesmo resultado em 278 e 275 dos 290 folds). No pool de Perceptrons a combinação suave é pior em média (-0.0046), mas a diferença **não é estatisticamente significativa** ($p = 0.51$). A conclusão que se sustenta é a negativa: para pools de classificadores lineares a média de margens normalizadas não é o método real a bater — o MVR é.

(c) O eixo que organiza a seleção dinâmica é a poda. O resultado mais robusto do experimento não é sobre a folga, é sobre o custo de descartar membros do pool. Testando cada fusor contra o MVR por base ($N = 29$), com correção de Holm para a família de 13 testes:

Tabela 6: Fusores contra o MVR no pool Random Forest (Wilcoxon pareado, Holm).

Fusor	Média	Δ *vs* MVR	Vitórias/derrotas	p (Holm)
LCA	0.7863	-0.0686	0/29	< 0.001
MCB	0.7900	-0.0649	0/29	< 0.001
Rank	0.7908	-0.0641	0/29	< 0.001
OLA	0.7915	-0.0634	0/29	< 0.001
Melhor individual	0.7958	-0.0592	1/28	< 0.001
KNORA-E	0.8274	-0.0275	3/24	0.0004
DES-KNN	0.8492	-0.0057	10/18	0.097
KNORA-U	0.8564	$+0.0015$	19/6	0.31
DES-P	0.8566	$+0.0017$	13/9	0.84
KNOP	0.8556	$+0.0006$	13/12	1.00
META-DES	0.8545	-0.0005	13/15	1.00
Seleção estática	0.8546	-0.0003	14/12	1.00
Combinação suave	0.8546	-0.0003	1/2	1.00

Em Bagging o padrão é idêntico: OLA, LCA, MCB e Rank perdem em 27–28 das 28 bases, com Δ entre -0.044 e -0.059 e $p_{\text{Holm}} \leq 0.0001$. A escala é clara:

- podar até *um* classificador (OLA, LCA, MCB, Rank, melhor individual) → perda de 0.044 a 0.069 nos pools de árvores, sempre significativa;
- podar parcialmente (KNORA-E) → perda intermediária, ainda significativa;
- não podar, apenas reponderar (KNORA-U, DES-P, META-DES, KNOP, seleção estática) → empate técnico com o MVR.

Os dois métodos estáticos confirmam o eixo pelos extremos: a seleção estática, que mantém o pool inteiro, empata com o MVR nos três modos; o melhor individual, que é poda máxima sem nenhuma localidade, está entre os piores em todos.

(d) Onde a seleção dinâmica de fato paga. No pool de Perceptrons a ordem se inverte parcialmente: **KNORA-U é o único fusor que supera o MVR de forma estatisticamente sustentada em algum modo** ($+0.0272$, $p_{\text{Holm}} = 0.0045$, vencendo em 147 dos 290 folds contra 49 derrotas e 94 empates). Nos pools de árvores o mesmo método fica em $+0.0001$ e $+0.0015$, sem significância.

A vantagem se concentra nas bases de fronteira não-linear em baixa dimensão: em P2, Lithuanian e Banana a recuperação média do PGDCS-F1/T1 é 0.766, contra 0.157 do Bagging e 0.155 do Random Forest nas mesmas bases. O mecanismo é geométrico: um Perceptron não separa a espiral do P2, mas separa qualquer vizinhança dela, e escolher o Perceptron certo por região é exatamente o que falta. Uma árvore já resolve a fronteira sozinha, então o voto majoritário já está perto do que o pool consegue e não sobra folga *local* a extrair.

(e) O viés de estimação foi medido, não apenas discutido. Numa versão anterior do protocolo o DSEL da seleção dinâmica era a mesma partição que o PGDCS-F1/T1 usa na função de *fitness*, o que favorecia o PGDCS-F1/T1 por construção. O protocolo atual separa as duas partições. Isolar o viés exige separar dois efeitos simultâneos — o viés removido (só afeta o PGDCS-F1/T1) e o DSEL pela metade (afeta os três modos) — e Bagging e RF servem de grupo de controle, porque mantêm os pools *bit-idênticos* entre os dois desenhos (verificado: $\max|\Delta| = 0.000000$ em toda coluna de pool e curva Oracle idêntica em 290/290 folds de cada modo).

Pela diferença-em-diferenças, restrita aos cinco métodos comuns aos dois desenhos: o controle perdeu -0.0057 de folga recuperada, o PGDCS-F1/T1 perdeu -0.0003, logo o viés é $+0.0055$ — cerca de 3% relativos, e nenhum dos deltas é significativo. O desenho anterior **não** estava inflando materialmente a vantagem do PGDCS-F1/T1; a correção está feita e é a que vale, mas não muda conclusão alguma.

2.7 Objetivo 7 — níveis intermediários são um limite mais realista?

Analisar se níveis intermediários de Oracle podem representar estimativas de limite superior mais conservadoras e mais próximas do desempenho de métodos reais.

A resposta é não, e o experimento identifica a razão estrutural.

Defina N^* como o menor N tal que $\text{Oracle}_N <$ votação majoritária — o nível em que o Oracle deixa de ser otimista.

Tabela 7: N^* por modo, com $M = 100$.

Modo	Mediana	Média
PGDCS completo	52	51.9
PGDCS-F1/T1	52	51.9
Bags aleatórios	52	52.0
Random Forest	53	53.0
Bagging	53	54.4

Friedman $p = 0.48$ ($k = 5$): **os cinco modos têm o mesmo** N^*, e ele fica em $N \approx M/2$. Não é coincidência empírica. O mecanismo não é a identidade $\text{Oracle}_{M/2+1} \equiv MVR$ — essa afirmação, feita em versões anteriores, é falsa — mas um *sanduíche*:

$$\text{Oracle}_{M/2+1} \;\leq\; MVR \;\leq\; \text{Oracle}_{M/2}. \tag{4}$$

A votação acerta sempre que mais da metade do pool acerta, logo $MVR \geq \text{Oracle}_{51}$; e ela só pode ganhar, além disso, os empates 50–50, logo nunca passa de Oracle_{50}. O sanduíche foi verificado em 1100 **de** 1100 folds binários (as 22 bases binárias $\times$ 10 folds $\times$ 5 modos), mas a igualdade $MVR = \text{Oracle}_{51}$ vale em apenas 833 deles (75,7%): nos demais 24,3% os empates são desfeitos a favor, o que empurra N^* acima de 51.

A distinção não é um detalhe — é ela que dá o valor de N^*. Como N^* é o primeiro nível *estritamente* abaixo do MVR, e $\text{Oracle}_{51} = MVR$ não satisfaz "$<$", os empates empurram N^* de 51 para 52 ou mais, que é exatamente onde as medianas caem.

O limite é do caso binário. 22 das 29 bases são binárias. Nas 7 multiclasse a votação é por pluralidade e um classificador correto não precisa de 51 votos para vencer, então o sanduíche não vale — e N^* desce de forma significativa:

Tabela 8: N^* mediano por cardinalidade de classes, cinco modos.

grupo	PGDCS	F1/T1	Bags aleat.	Bagging	RF
binárias (22)	53.1	53.4	53.5	53.7	53.5
multiclasse (7)	45.7	44.4	44.3	48.1	47.3

Mediana global 53.6 contra 47.3, Mann-Whitney $p = 2.2 \times 10^{-11}$. Dentro de cada grupo os cinco modos continuam sem diferença estatística significativa.

Disso decorre a resposta ao objetivo, **restrita aos problemas binários**:

- Para $N \ll M/2$ a curva está **saturada**: os cinco modos ficam entre 0.988 e 0.998 em $N = 1..5$, e $\text{Oracle}_5 \geq MVR$ em praticamente todos os folds. As diferenças entre modos são estatisticamente reais em alguns pares ($p < 0.001$) mas de magnitude 0.001–0.004. Mais conservador que Oracle_1, sim; perto de um método real, não.
- Para $N \approx M/2$ a curva **já é** a votação majoritária, sob outro nome. Não acrescenta informação.
- **Não existe faixa intermediária útil** entre as duas.

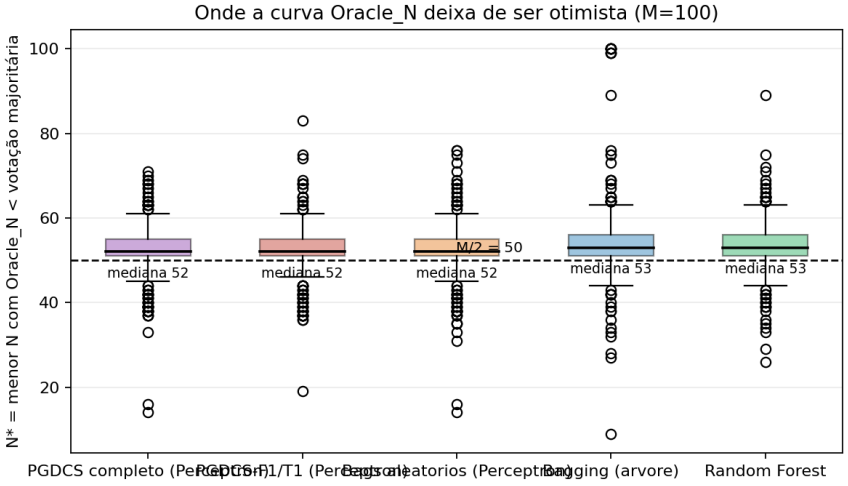


Figura 4: Distribuição de N^*. A concentração em torno de $M/2$ nos cinco modos é o que impede qualquer nível intermediário de servir como limite realista.

O que substitui a ideia original. O objetivo 7 buscava uma estimativa conservadora e alcançável. Ela existe, mas não é "Oracle_N para algum N"; é a folga $\text{Oracle}_1 - MVR$ ponderada pela fração dela que métodos reais recuperam. Medido: mesmo tomando o melhor de dez métodos dinâmicos escolhido *a posteriori* no teste, a recuperação média fica em 24,3% (PGDCS completo), 24,0% (PGDCS-F1/T1), 23,1% (bags aleatórios), 15,0% (Bagging) e 14,2% (RF) — os três pools de Perceptron recuperam quase o dobro dos pools de árvore, consistente com o achado do objetivo 4 de que o classificador-base, não o método de geração, organiza os resultados. As medianas são bem menores em todos os modos (0.167 nos três de Perceptron; 0.108 e 0.091 em Bagging e RF), e um quarto dos folds de árvore recupera *exatamente zero* (31,1% no agregado).

Correção. Versões anteriores apresentavam

$$MVR + 0.15\,\big(\text{Oracle}_1 - MVR\big) \tag{5}$$

como "limite superior operacional". Não é um limite: 0.15 é a recuperação *média* dos pools de árvore, e uma média é ultrapassada por metade da massa. Medido nos folds de árvore, 37,7% deles recuperam mais que 0.15. A expressão é uma **estimativa**, $\widehat{\text{Acc}}_{DS}$, e deve ser lida como tal.

Quem quiser de fato uma cota precisa de um quantil alto da distribuição de R = recuperação, não da sua média:

$$\widehat{\text{Acc}}_{DS} = MVR + \overline{R}\,\big(\text{Oracle}_1 - MVR\big), \qquad \text{cota}_{95} = MVR + Q_{0.95}(R)\,\big(\text{Oracle}_1 - MVR\big), \tag{6}$$

com $Q_{0.95}(R) = 0.50$ nos pools de árvore — mais de três vezes a média. Ambos permanecem otimistas, porque R é medido escolhendo o melhor método no próprio teste.

Robustez à métrica. A metodologia do subprojeto pede precisão, revocação, F1 e acurácia balanceada além da acurácia — a preocupação legítima sendo que a acurácia bruta esconda a classe minoritária, num catálogo que chega a IMB 71.5. As quatro métricas foram gravadas para *todo* fusor, nos cinco modos. Em acurácia balanceada tudo cai, mas de forma quase uniforme, e a folga *cresce*:

Tabela 9: Acurácia *versus* acurácia balanceada.

Modo	Oracle_1	Oracle_1 balanc.	MVR	MVRbalanc.	Folga	Folga balanc.
PGDCS completo	0.9972	0.9961	0.7771	0.7349	0.2201	0.2611
PGDCS-F1/T1	0.9976	0.9967	0.7784	0.7368	0.2192	0.2599
Bags aleatórios	0.9979	0.9970	0.7808	0.7390	0.2170	0.2580
Bagging	0.9951	0.9923	0.8467	0.8163	0.1484	0.1760
Random Forest	0.9970	0.9949	0.8549	0.8214	0.1421	0.1735

O Oracle_1 é quase insensível ao desbalanceamento — basta *um* classificador do pool acertar a instância minoritária — enquanto o MVR precisa que 51 acertem, e é aí que a minoria se perde. Nos *ranks* de fusor a única mudança sistemática é o DCS puro melhorar posições nos pools de árvore, o que faz sentido — escolher o competente na vizinhança ajuda numa região minoritária. Mas em nenhum modo isso muda o topo: KNORA-U, META-DES, MVR e combinação suave continuam empatados na frente, e OLA e LCA continuam no fundo. **Nenhuma conclusão deste relatório depende da escolha da métrica.**

2.8 Objetivo 8 — recomendações e redução de custo

Gerar recomendações sobre o uso de Oracles em diferentes níveis como ferramenta de análise prévia do potencial de desempenho de ensembles, considerando a redução de custo computacional.

A base quantitativa da recomendação é que o índice de redundância prevê a folga *antes* de treinar qualquer método de seleção:

$$\text{Spearman}\big(\text{DF}/e^2,\ \text{Oracle}_1 - MVR\big) = -0.887, \quad p = 8 \times 10^{-50}, \quad n = 145. \tag{7}$$

Por modo: PGDCS-F1/T1 -0.933, bags aleatórios -0.929, PGDCS completo -0.920, Bagging -0.909, RF -0.785. O índice custa $O(M^2 n)$ sobre a matriz de acertos que já se tem.

As oito recomendações que o experimento sustenta:

1. **Use $\text{Oracle}_1 - MVR$ como medida de potencial, não Oracle_1 como meta.** O Oracle_1 fica entre 0.995 e 0.998 nos cinco modos e 29 bases: não discrimina. A folga discrimina (0.142 a 0.220 entre modos; maior ainda entre bases).

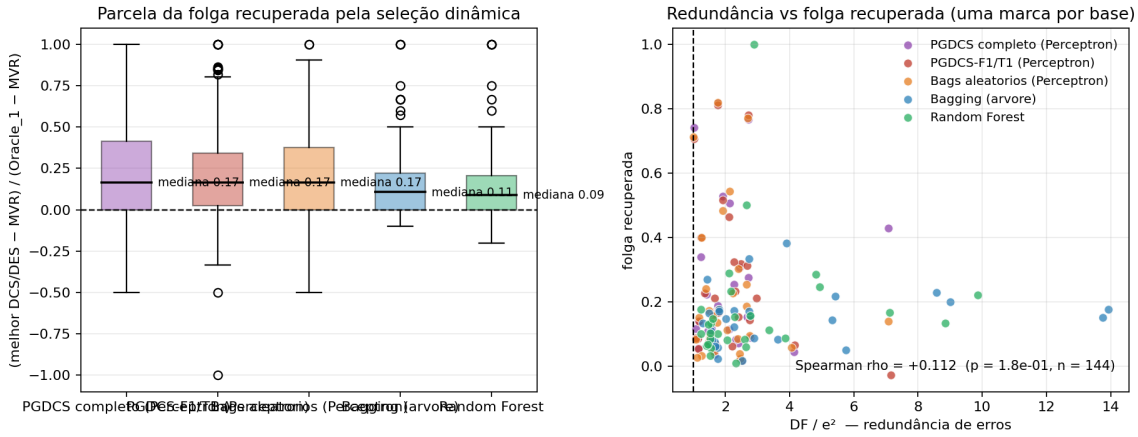


Figura 5: À esquerda, a distribuição da folga recuperada por modo. À direita, a mesma quantidade contra DF/e^2: a correlação que prevê o *tamanho* da folga não prevê a *fração alcançável*.

2. **Não procure um Oracle$_N$ intermediário como limite realista.** N^* fica em $N \approx M/2$ nos cinco modos (Friedman $p = 0.48$). Reporte Oracle$_1$, a folga e N^* como diagnóstico; não reporte um nível intermediário como cota.

3. **Se o classificador-base for Perceptron, o PGDCS completo não compensa o custo.** Reativar a votação de medidas de complexidade (`get_best_types`) custou 10,7h de relógio a mais, concentradas em duas bases, e produziu um pool estatisticamente indistinguível do que as medidas fixas F1/T1 já entregavam em Oracle$_1$, MVR, folga, DF/e^2 e recuperação (nenhum $p < 0.29$). A votação também raramente escolhe F1/T1 (1,4% dos folds) — o atalho não acerta a escolha, mas empata o resultado. Use PGDCS-F1/T1 em produção; reserve o PGDCS completo para quando a hipótese em teste for exatamente sobre a escolha de medidas.

4. **Calcule DF/e^2 antes de treinar qualquer método de seleção.** Regra operacional medida neste experimento:

DF/e^2	Folga típica	Decisão
≥ 4	< 0.05	não vale DCS/DES — o custo não se paga
2–4	0.05–0.15	marginal; só KNORA-U ou DES-P, sem esperar ganho
≈ 1	> 0.30	candidato forte, sobretudo com base fraco

Os cortes foram validados fora da amostra, porque descobri-los e avaliá-los nas mesmas 29 bases seria circular. Reajustando os dois limiares em 28 base×modo e prevendo o 29º, 145 vezes: acerto 0,834 contra 0,759 da regra trivial, e os limiares reajustados caem na mediana em 1,23 e 4,82 — praticamente os ≈ 1 e ≥ 4 da tabela. A regra generaliza.

Sobre o denominador. e^2 com e médio só é o valor esperado sob independência se todos os classificadores errarem na mesma taxa; o exato é $\frac{2}{M(M-1)} \sum_{i<j} e_i e_j$. Recalculado a partir das acurácias individuais já gravadas por fold, o índice exato desloca a correlação de -0.8869 para -0.8878 e melhora a validação fora da amostra de 0,834 para 0,855, com os limiares caindo em 1,23 e 4,16. A razão entre os dois índices tem mediana 1,0006, porque o desvio médio das acurácias individuais dentro de um pool é pequeno (0,064). A correção melhora o rigor e, desta vez, também a validação fora da amostra — não é só estética.

5. DF/e^2 **não prevê sozinho o que você vai recuperar, mas complexidade de dados prevê melhor que o acaso.** Um modelo linear (*ridge*) sobre medidas de complexidade,

n_{classes}, dimensão, desbalanceamento, DF/e^2 e acurácia individual média falha fora da amostra ($R^2_{\text{LODO}} = -0{,}018$, pior que prever a média). Uma floresta aleatória sobre as mesmas variáveis generaliza ($R^2_{\text{LODO}} = +0{,}355$, $\rho = +0{,}335$, $p = 4 \times 10^{-5}$, EAM 0,109 contra 0,141 do *baseline* constante) — a relação entre complexidade e recuperabilidade não é linear, mas existe e é aprendível com $n = 29$ bases via validação leave-one-dataset-out.

6. **Se for usar seleção dinâmica, use um método que não pode o pool.** A fração média selecionada por consulta ($E[S]/M$, medida diretamente sobre `ds.select`, não inferida) correlaciona-se com a margem sobre o MVR ($\rho = 0{,}54$, $n = 1566$, $p < 10^{-4}$; mais forte nos pools de árvore — RF $\rho = 0{,}80$, Bagging $\rho = 0{,}74$ — que nos de Perceptron, $\rho \approx 0{,}39$ a 0,41). OLA, LCA, MCB e Rank selecionam 1% do pool por consulta e perdem de 0,044 a 0,069 contra o MVR nos pools de árvore, em 27–29 das 29 bases, com $p_{\text{Holm}} < 0.001$. KNORA-U (100% do pool, ponderado) é o único que supera o MVR de forma sustentada em algum modo e nunca perde em nenhum.

7. **Em pool homogêneo forte (Bagging, Random Forest) a decisão padrão é não usar DCS/DES.** Nenhum dos 13 fusores supera a votação majoritária de forma significativa em nenhum dos dois modos. O melhor caso é +0.0019 (KNOP em Bagging), que não sobrevive a Holm. O MVR é grátis; os outros não.

8. **O custo foi medido, não estimado, em toda a escala do desenho.** Passar de 5 para 12 métodos de seleção levou de 2h21 para 4h08 nas mesmas 29 bases × 10 folds × 3 modos — fator 1,76×. Acrescentar dois modos de controle (bags aleatórios, PGDCS completo) ao desenho de cinco levou o run canônico completo a 16h44 em 1450 folds, dos quais 10,7h (64%) foram só a votação de medidas do PGDCS completo — o custo que a recomendação 3 mostra não se pagar em resultado.

3 Replicação do PGDCS contra a fonte

O objetivo 4 concluiu, dentro do protocolo deste projeto, que votar as medidas de complexidade não muda Oracle_1, MVR nem DF/e^2 de forma sustentada, e o objetivo 8 traduziu isso numa recomendação: o PGDCS completo não paga o custo que cobra. A fonte do método relata o oposto — ganhos de 4 a 6 pontos percentuais sobre Bagging na Tabela A.10 de Monteiro et al. (2023). Duas afirmações incompatíveis sobre o mesmo método exigem reconciliação, e ela só é possível replicando o protocolo do artigo, não o deste relatório.

Esta seção reporta essa replicação. Ela usa árvore de decisão (não Perceptron) e duas configurações de protocolo registradas na ADR 0020, executadas em `7c5ec6d`:

Perfil do artigo. `monteiro-2023-dt-compatible-v1`: 20 *holdouts* estratificados independentes 50/25/25, semente-raiz 86, validação *coincidente* com o DSEL, $M = 100$, 20 gerações. Run `2026-08-27T19-47-09`: 1620 linhas, 27 bases, 3h14.

Perfil do POS. `pos-oracle-v2`: 10-fold estratificado com partição interna treino / validação / DSEL *disjunta*, semente 42, mesmo indutor e mesmo M. Run `2026-08-27T23-01-32`: 840 linhas, 28 bases, 5h38.

3.1 O gap, e o que o produz

Contra a Tabela A.10, os três tratamentos se comportam de modo qualitativamente diferente:

O Bagging — que compartilha *split*, semente, indutor, M e protocolo de avaliação com os outros dois — erra 0,75 pp em módulo, com média com sinal de −0,16 pp e 14 negativos em 27: ruído simétrico, a assinatura de uma reimplementação fiel. O PGDCS erra sempre para baixo, em 54 comparações consecutivas, sem uma exceção. Um defeito de implementação dispersaria nos dois sentidos; um viés sistemático, não.

Tabela 10: Diferença contra a Tabela A.10 no perfil compatível com o artigo, 27 bases. O que decide a questão é o *sinal*, não a magnitude.

Tratamento	erro médio $\lvert\Delta\rvert$	média com sinal	negativos
PGDCS_A	6,19 pp	**−6,19** pp	**27/27**
PGDCS_D	5,25 pp	**−5,25** pp	**27/27**
Bagging	0,75 pp	−0,16 pp	14/27

A causa é viés de seleção na medida publicada. O Algoritmo 1 do artigo escolhe a geração final por $v \leftarrow \text{acurácia}(E^{(t+1)}, V)$, e a Tabela A.10 reporta essa mesma quantidade — o máximo sobre 20 gerações, medido no *mesmo* conjunto de 25% que fez a escolha. O número do Bagging, que não passa por seleção alguma, é uma medida honesta em teste. Gravando as trajetórias geração a geração em Wine, Heart, Blood e Sonar (20 replicações cada), a distinção fica direta: chamando `VAL@sel` a acurácia da geração escolhida no conjunto de validação e `TEST@sel` a da mesma geração em teste, o publicado dista $\approx 1{,}7$ pp de `VAL@sel` e $\approx 6{,}3$ pp de `TEST@sel`.

Três verificações independentes fecham o diagnóstico:

- **A trajetória tem a forma de sobreajuste ao conjunto de seleção.** A MVR de validação sobe monotonicamente até a geração 20; a acurácia de teste sobe até a geração $\approx$ 4–8 e depois cai (Wine: validação 95,0 → 99,8, teste 95,3 → 95,0).
- **O teto por replicação não alcança o publicado.** Tomando, em cada replicação, a *melhor* geração medida em teste — um oracle sobre gerações, inatingível por qualquer critério de parada — Wine chega a 98,0/97,4, ainda abaixo dos 99,3 publicados. Nenhuma política de seleção fecharia o gap.
- **O que não depende do critério de saída reproduz.** Oracle_1 bate a Tabela 7 com erro médio de 0,44 pp e a geração escolhida bate a Tabela 6 com 3,19 gerações de erro. O Oracle reproduz porque não passa pela seleção.

3.2 O tamanho honesto do efeito

Com os dois perfis executados, dá para medir o que o PGDCS ganha do Bagging quando as duas pontas são medidas em teste — o contraste que a Tabela A.10 não faz:

Tabela 11: PGDCS_A menos Bagging, acurácia MVR em teste, pareado por base (Wilcoxon). O ganho existe, é uma ordem de grandeza menor que o publicado, e depende do tamanho do conjunto de validação.

Perfil	Validação	Δ médio	Vence em	p
`monteiro-2023-dt`	25%, = DSEL	**+0,39** pp	22/27	**0,0030**
`pos-oracle-v2`	10%, disjunta	−0,13 pp	12/27	0,75

No perfil compatível com o artigo o ganho existe e é consistente, mas vale $\approx 0{,}4$ pp contra os 4–6 pp anunciados. No perfil deste projeto, cuja validação é menor e disjunta do DSEL, ele desaparece: 12 vitórias em 27 é sorteio ($p = 0{,}75$). A seleção de geração precisa de um conjunto de validação grande para não escolher ruído, e o que entrega, quando entrega, é marginal.

Isto confirma a conclusão do objetivo 4 por um caminho independente. Lá, o PGDCS completo era indistinguível dos bags aleatórios em pools de Perceptron sob o protocolo do POS; aqui, com árvore de decisão e sob o protocolo do próprio artigo, o ganho publicado se decompõe em viés de seleção mais 0,4 pp reais. As duas medições não estavam em conflito com a fonte — a fonte é que compara uma quantidade otimizada com uma não otimizada.

Nenhuma mudança de protocolo foi feita para fechar o gap , porque fechá-lo exigiria reportar a métrica enviesada. A coluna comparável à Tabela A.10 (MVR de validação da geração vencedora) fica gravada no manifesto de fold como `pgdcs_best_score`; `mv_acc` continua sendo acurácia em teste.

3.3 Duas correções de implementação encontradas no caminho

A replicação expôs dois defeitos no PGDCS deste projeto, ambos corrigidos por TDD e sem efeito sobre o diagnóstico acima:

Votação das medidas (Eq. 10–11). O voto usava a *média acumulada* dos σ_j^t em vez do σ_j^t da própria iteração, o que concentrava 9 ou 10 dos 10 votos de uma família numa única medida — contra o espalhamento que as Tabelas 3 e 4 do artigo mostram. Corrigido, o acerto contra a Tabela 4 sobe de 1/5 bases para 4/8 medidas nas quatro primeiras bases. O efeito na acurácia é de +0,6 a +1,3 pp, só no PGDCS: a escolha das medidas não é a causa do gap.

A fração u. ğ5.1 e ğ5.2 do artigo descrevem passos diferentes na mesma frase. Adotado $u = 0{,}4$ para as sub-amostras da votação e 0,5 para os *bags* do AG; antes ambos valiam 0,5.

4 Confronto com os resultados esperados

Esperado no projeto	**Obtido**
Caracterizar quantitativamente Oracle_N em múltiplos níveis e construir a curva.	Confirmado. Curva completa $N = 1..M$ por fold, monotonicidade em 1450/1450, e a identidade que liga a média da curva à acurácia individual.
Verificar se níveis intermediários se aproximam de métodos práticos.	**Refutado**, com mecanismo identificado: $N^* \approx M/2$ porque $\text{Oracle}_{M/2+1} \leq MVR \leq \text{Oracle}_{M/2}$ em problema binário. Abaixo disso a curva está saturada. Resultado negativo, mas informativo — fecha a pergunta em vez de deixá-la em aberto.
Implementação reprodutível de $\text{Oracle}_1..\text{Oracle}_M$, com scripts de treino, matriz de acertos, métricas e gráficos.	Entregue. Manifesto por fold com *git SHA*, versões de dependências e sementes; 8 figuras e testes estatísticos gerados por um único comando — e desta vez a partir de árvore limpa, com o run anterior validado bit a bit contra ele.
Relacionar diversidade, redundância de acertos e desempenho.	Entregue e quantificado: DF/e^2 prevê a folga com $\rho = -0.89$ ($n = 145$), validado fora da amostra (85,5% de acerto *leave-one-dataset-out*), e o mesmo índice sozinho *não* prevê a fração recuperável — mas complexidade de dados, via floresta aleatória, prevê ($R^2_{\text{LODO}} = 0.36$). Nenhuma distinção estava antecipada no projeto.

Esperado no projeto	Obtido
Indicar em que cenários o Oracle tradicional superestima o desempenho alcançável.	Entregue: superestima sempre, e mais ainda em pools diversos de base fraco. Entre 76% e 86% da folga anunciada não é recuperada nem pelo melhor de dez métodos escolhido no teste, e essa faixa é explicada pelo classificador-base — não pelo método de geração do pool, um confounder que o desenho atual separa explicitamente.
Apoiar a decisão sobre usar métodos complexos de seleção.	Entregue como as oito recomendações operacionais, com um limiar numérico em DF/e^2 validado fora da amostra, um modelo preditivo de recuperabilidade e a constatação de que o PGDCS completo não compensa o custo adicional sobre a variante de medidas fixas.
Não previsto no projeto: confrontar o PGDCS com os números publicados na fonte que o propõe.	Feito, e reconciliado. Sob o protocolo do próprio artigo, o PGDCS erra a Tabela A.10 para baixo em 27/27 bases nos dois critérios de saída, enquanto o Bagging — mesmo *split*, semente e indutor — erra 0,75 pp com sinal simétrico. A causa é viés de seleção, não implementação: o valor publicado é o máximo de 20 gerações medido no mesmo conjunto que escolheu a geração. Medido em teste, o ganho real é +0,39 ponto. Duas correções de implementação saíram do processo (votação das medidas, fração u).

5 Limitações

- **Bases de imagens.** O objetivo 5 as menciona como preferência; o catálogo atual privilegia comparabilidade com a literatura de referência. É a pendência mais direta.
- $M = 100$ **fixo.** Todas as conclusões sobre N^* são relativas a $M = 100$. O argumento estrutural ($\mathrm{Oracle}_{M/2+1} \leq MVR \leq \mathrm{Oracle}_{M/2}$ em problema binário) é independente de M, mas a verificação empírica não varreu M.
- **Dois métodos ausentes nos pools de Perceptron.** META-DES e KNOP exigem `predict_proba`, indisponível no Perceptron linear — vale para os três modos que o usam (PGDCS completo, PGDCS-F1/T1, bags aleatórios). As colunas ficam vazias com o motivo gravado por fold, e as comparações entre modos que os envolvem foram restritas aos modos aplicáveis — não houve imputação.
- **A instrumentação de poda mede só as consultas roteadas.** O DESlib curto-circuita amostras em que a vizinhança é unânime, e essas nunca chegam ao seletor. $E[S]/M$ é medido sobre as consultas roteadas para seleção dinâmica (fração gravada junto, entre 76% e 100% conforme o método); não é uma medida sobre todas as consultas.
- **"Melhor por fold" é otimista.** A métrica de folga recuperada usa o máximo entre métodos escolhido no conjunto de teste. Um seletor de método honesto entregaria menos. Os números de recuperação são cotas superiores.

- **Duas bases ficam fora da comparação com a Tabela A.10.** Em Phoneme, o número publicado do Bagging (93,1) está acima do teto do *snapshot*: com 90% de treino em vez dos 50% do protocolo e Random Forest no lugar de Bagging, o máximo alcançável é 91,1% — e os valores medidos batem com a literatura da base (árvore 87,4; RF e 1-NN ≈ 90–91). Não é `max_samples` (mover de 0,5 para 1,0 vale 0,7 pp) nem a base (o *snapshot* bate com a Tabela 2 e com o Phoneme canônico da ELENA). Sem Phoneme, o erro do Bagging cai de 0,75 para 0,59 pp médio, com máximo de 2,27 pp. P2 fica fora por motivo diferente: o *snapshot* tem 2000 instâncias contra 5000 geradas no artigo, então entra no `pos-oracle-v2` — que não replica ninguém — mas não na comparação publicada.

- **O portão de conformance é global.** Ele aplica a Tabela 2 de Monteiro et al. (2023) a qualquer perfil, inclusive ao `pos-oracle-v2`, cujo escopo de referência é ele mesmo. P2 só entrou porque foi passado `-allow-unverified-data`, o que marca o run inteiro como não comparável. Fazer o portão consultar o escopo do perfil é pendência registrada.

- **Poder estatístico com** 14 **fusores.** A diferença crítica de Nemenyi cresce com o número de colunas a ponto de nada ser significativo. Por isso o protocolo mantém um conjunto primário de 7 métodos para Friedman/Nemenyi e trata os demais em camada descritiva, com Wilcoxon pareado contra o MVR e correção de Holm.

Referências

CRUZ, R. M. O.; SABOURIN, R.; CAVALCANTI, G. D. C. Dynamic classifier selection: Recent advances and perspectives. *Information Fusion*, v. 41, p. 195–216, 2018.

CRUZ, R. M. O.; SABOURIN, R.; CAVALCANTI, G. D. C. META-DES.Oracle: Meta-learning and feature selection for dynamic ensemble selection. *Information Fusion*, v. 38, p. 84–103, 2017.

DEMŠAR, J. Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research*, v. 7, p. 1–30, 2006.

KUNCHEVA, L. I.; WHITAKER, C. J. Measures of Diversity in Classifier Ensembles and Their Relationship with the Ensemble Accuracy. *Machine Learning*, v. 51, p. 181–207, 2003.

MONTEIRO JR., M.; BRITTO JR., A. S.; BARDDAL, J. P.; OLIVEIRA, L. S.; SABOURIN, R. Exploring diversity in data complexity and classifier decision spaces for pool generation. *Information Fusion*, v. 89, p. 567–587, 2023.

SOUZA, M. A.; CAVALCANTI, G. D. C. On the Characterization of the Oracle for Dynamic Classifier Selection. In: *IJCNN*, 2017, Anchorage. p. 332–339.